

Software engineering

C# and projects

Skill progressions by applying material:

C# + .NET +3

Logistics

- I've updated the lab rules saying that a team must come in their own lab slot, so we have time for everybody
- Minor corrections to lab rules/assignments can be expected (typos, me not being clear enough, exploits etc.)
- If you are inclined to live-blog/video your experience, I'll award a team lead bonus to you (max 1/team), I'd suggest that somebody other than the team lead do this.



C# .NET

```
// C# is strongly typed
int age = 25;
float temperature = 98.6f;
double distance = 1200.78;
string name = "John Doe";
char grade = 'A';
bool isStudent = true;
decimal accountBalance = 1000.50;
int? age = null;
// No global variables
// And you have garbage collection
```

```
private enum CharacterTypeEnum
{
    Wizard = 100,
    Fighter = 200,
    Thief = 300
}
```

```
internal struct Location
{
    public double Latitude { get; set; }
    public double Longitude { get; set; }
}

protected class Calculate // Single inheritance
{
    public int Add(int number1, int number2)
    {
        return number1 + number2;
    }
}

public interface IDrawable // Multiple inheritance
{
    void Draw();
}
```

```
int[] numbers = { 1, 2, 3, 4, 5 };

int[,] twoDimensional = new int[3, 3]
{
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};

int[][] jagged = new int[3][]
{
    new int[] { 1, 2, 3 },
    new int[] { 4, 5, 6, 7 },
    new int[] { 8, 9 }
};

List<int> list = new List<int> { 1,
2, 3, 4, 5 };
```

```
Dictionary<string, int> ages =
    new Dictionary<string, int>();
ages["Alice"] = 25;
ages["Bob"] = 30;

HashSet<int> uniqueNumbers = new HashSet<int>();
uniqueNumbers.Add(5);

Queue<string> waitingQueue = new Queue<string>();
waitingQueue.Enqueue("Customer 1");
string nextCustomer = waitingQueue.Dequeue();

Stack<int> numberStack = new Stack<int>();
numberStack.Push(10);
int poppedNumber = numberStack.Pop();

public class Node<T>
{
    public T Data { get; set; }
    public Node<T> Next { get; set; }
}
```

```
string name = "John";
```

```
int age = 30;
```

```
Console.WriteLine("Hello " + name + " you are " + age + " years old.");
```

```
Console.WriteLine(string.Format("Hello {0} you are {1} years old.", name, age));
```

```
Console.WriteLine($"Hello {name} you are {age} years old.");
```

```
if (string.IsNullOrEmpty(student.Group))
```

```
StringBuilder builder = new StringBuilder();
```

```
builder.Append("Hello");
```

```
builder.Append(" World");
```

```
string result = builder.ToString();
```

```
int result = 10 + 5; // Addition
float discount = 100.0f - 25.0f; // Subtraction
double totalPrice = 20.5 * 4; // Multiplication
int quotient = 15 / 3; // Division
int remainder = 10 % 3; // Modulo

bool isEqual = (5 == 5); // Equality
bool notEqual = (10 != 5); // Inequality
bool greaterThan = (15 > 10); // Greater Than
bool lessThan = (7 < 10); // Less Than

bool logicalAnd = (true && false); // AND
bool logicalOr = (true || false); // OR
bool logicalNot = !true; // NOT

int x = 10;
x += 5; // x = x + 5
x++; // x = x + 1
```

```
int age = 22;
if (age >= 18)
{
    Console.WriteLine("You're eligible to vote.");
}
else
{
    Console.WriteLine("You're not eligible to vote yet.");
}

string country = "France";
switch (country)
{
    case "Germany":
        Console.WriteLine("Capital: Berlin");
        break;
    case "France":
        Console.WriteLine("Capital: Paris");
        break;
    default:
        Console.WriteLine("Capital: Not Available");
        break;
}
```

```

for (int i = 0; i < 5; i++)
{
    Console.WriteLine("For: " + i);
}

int[] numbers = { 3, 6, 9, 12 };
foreach (int number in numbers)
{
    Console.WriteLine("Foreach: " + number);
}

int j = 0;
while (j < 5)
{
    Console.WriteLine("While: " + j);
    j++;
}

do
{
    Console.WriteLine(
        "This runs at least once.");
} while (false);

```

```

ValueType : ObjectType
int valueType = 42;
object objectType = valueType;
int unboxed = (int)objectType;

#region boring code
// this part is collapsible
#endregion

Delegates
Async/Await

```

```
var numbers = new int[] { 5, 10, 8, 3,6};
```

```
//(SELECT number FROM numbers WHERE  
number % 2 = 0 ORDER BY number)
```

```
// Query syntax
```

```
var orderedEvenNumbers1 =  
    from number in numbers  
    where number % 2 == 0  
    orderby number  
    select number;
```

```
// Method syntax
```

```
var orderedEvenNumbers2 =  
    numbers  
    .Where(number => number % 2 == 0)  
    .OrderBy(number => number);
```

```
public static int ReverseDigits(this int number)  
{  
    char[] digits =  
        number.ToString().ToCharArray();  
    Array.Reverse(digits);  
    string newDigits = new string(digits);  
    return int.Parse(newDigits);  
}
```

```
int myInt = 12345678;  
int reversed = myInt.ReverseDigits();
```

```
public static Box operator +(Box first, Box second)
{
    Box box = new Box();
    box.length = first.length + second.length;
    box.width = first.width + second.width;
    box.height = first.height + second.height;
    return box;
}
```

```
string greeting = DateTime.Now.Hour < 12 ? "Good Morning" : "Good Afternoon";
```